



Universidade do Minho
Escola de Engenharia

Sistema de Aquisição de sinais de um Acelerómetro

Instrumentação Avançada

Ano Letivo 2023/2024

Beatriz Alvadia PG53691

Daniela Longras PG53755

Docentes: Paulo Mendes e José Fernandes

Braga, 2 de janeiro de 2024

Resumo

Este projeto teve-se como principal objetivo o desenvolvimento de um sistema de aquisição e processamento de sinais provenientes de um acelerômetro durante o movimento de marcha do utilizador.

Posto isto, foi necessário incluir um acelerômetro *MMA7361L* e uma placa *Curiosity HPC* onde está inserido um microcontrolador *PIC18F47Q10*. Entre este último e o computador foi estabelecida uma comunicação série permitindo o envio de informação entre ambos. A comunicação foi efetuada utilizando um cabo USB, através da comunicação série *RS232* e foi tida em conta a norma IEEE Std. 1451. A programação do microcontrolador foi conseguida com recurso ao *software MPLAB*, utilizando tanto linguagem *C* como linguagem *Assembly*. Em *MATLAB* foi desenvolvido o algoritmo capaz de receber e enviar dados ao microcontrolador. Foi, ainda, concebida uma aplicação utilizando a ferramenta *App Designer* do *software MATLAB* para apresentar os resultados obtidos e promover a interação entre o utilizador e o sistema.

Foi efetivamente possível realizar uma comunicação entre o microcontrolador e o computador, permitindo que seja concretizado o envio de informação. Foi então possível o envio de informação relativa às METATEDS e TEDS para a aplicação. Assim como também o envio dos sinais dos três eixos do acelerômetro, podendo visualizar-se os respetivos gráficos ao longo do número de amostras e, ainda, os respetivos valores. Por fim, foi concretizável, para cada eixo, a ativação de um LED quando um certo valor de aceleração fosse ultrapassado.

Índice

1	Introdução	1
2	Especificações do Sistema	2
2.0.1	Diagrama de Blocos	3
2.0.2	Fluxograma	4
2.0.3	Fluxograma da Interface da Gráfica	6
3	Programação do Microcontrolador	7
3.1	<i>config</i>	7
3.1.1	Configuração PORTA	8
3.1.2	Configuração PORTC	8
3.1.3	Configuração CLOCK	8
3.1.4	Configuração USART	9
3.1.5	Configuração ADC	10
3.1.6	Configuração das Interrupções	11
3.2	<i>main</i>	12
3.2.1	<i>main</i>	12
3.2.2	<i>define_METATEDS</i>	12
3.2.3	<i>define_TCTEDS</i>	13
3.2.4	<i>identify_NCAP_cmd</i>	14
3.2.5	<i>send_METATEDS</i>	15
3.2.6	<i>send_TCTEDS</i>	15
3.2.7	<i>send_VALUES_SENSOR</i>	15
3.2.8	<i>receive_VALUES</i>	16
3.2.9	<i>putch</i>	16
3.2.10	<i>getch</i>	16
3.3	<i>interrupt_handler</i>	17
3.3.1	<i>Interrupção do ADC</i>	17
3.3.2	<i>Interrupção do RX</i>	17
3.4	<i>TEDS definition</i>	18

4	Interface	19
5	Resultados e Discussão	21
6	Conclusão	25
	Referências	26

1 Introdução

O presente trabalho, realizado no âmbito da Unidade Curricular de Instrumentação Avançada, teve como principal objetivo desenvolver um sistema de aquisição e processamento de sinais provenientes de um acelerómetro durante o movimento de marcha do utilizador.

Posto isto, foi necessário incluir um acelerómetro *MMA7361L* e uma placa *Curiosity HPC* onde está inserido um microcontrolador *PIC18F47Q10*. A programação do microcontrolador foi conseguida com recurso ao *software MPLAB*, utilizando tanto linguagem *C* como linguagem *Assembly*. A comunicação entre o microcontrolador e o computador foi efetuada utilizando um cabo USB, através da comunicação série *RS232* e foi tida em conta a norma IEEE Std. 1451. Em *MATLAB* foi desenvolvido o algoritmo capaz de receber e enviar dados ao microcontrolador, isto é, foi possível estabelecer-se comunicação entre a NCAP (*Network Capable Application Processor* e o TIM *Transducer Interface Module*. Foi, ainda, concebida uma aplicação utilizando a ferramenta *App Designer* do *software MATLAB* para apresentar os resultados obtidos e promover a interação entre o utilizador e o sistema.

Em suma, ao longo do presente relatório serão, primeiramente, abordadas as especificações do sistema, permitindo perceber como este funciona e que blocos estão incluídos. De seguida, exibe-se o propósito e conteúdo dos ficheiros *config.asm*, *main.c*, *interrupt_handler.asm* e *TEDS_definition.h*. No primeiro são exibidas as configurações efetuadas no microcontrolador, no segundo o desenvolvimento do código principal que é executado quando se processa o programa, no terceiro o desencadeamento de quando ocorre uma interrupção e no último a definição das estruturas das META-TEDS e das TEDS.

2 Especificações do Sistema

Para a elaboração de um sistema de aquisição e processamento de sinais provenientes de um acelerômetro durante o movimento de marcha do utilizador foi necessário incluir um acelerômetro *MMA7361L* e um microcontrolador *PIC18F47Q10* de 40 pins ligados entre si.

A TIM (*Transducer Interface Module*) é composta pelo microcontrolador *PIC18F47Q10* de 40 pins, pelo acelerômetro *MMA7361L* e pelo atuador (LED). Trata-se de um acelerômetro que mede a aceleração nos três eixos (x, y e z), pelo que se recebe três sinais e, por esse motivo, considerou-se três canais analógicos.

O sistema é desenvolvido com o intuito de ser utilizado durante o movimento de marcha do utilizador. Esse movimento humano alcança uma frequência máxima de 80Hz. Assim, compreendo a largura de banda 80Hz, pode definir-se a frequência de amostragem com base no Teorema de *Nyquist*. Este afirma que a frequência de amostragem deve ser igual ou superior a duas vezes a frequência máxima. Assim sendo, a frequência de amostragem para o sinal de cada eixo é de 160Hz. Como são considerados os três eixos, na verdade a frequência de amostragem total corresponde a 480Hz.

Tendo em conta as notas presentes no *datasheet* [1] referente ao acelerômetro *MMA7361L*, colocou-se um condensador de 0.1 μ F no V_{DD} para anular a energia da fonte. Em cada uma das saídas do acelerômetro, X_{OUT} , Y_{OUT} e Z_{OUT} , colocou-se um condensador de 3.3nF para minimizar o ruído do relógio e, ainda, resistências de 10M Ω ou mais para evitar perdas devido à relação de divisor de tensão entre a resistência interna de 32 k Ω e a impedância de entrada de medição. De notar que o *sleep mode* do se encontra ligado à saída 3.3V.

Na Figura 1, é demonstrada a ligação estabelecida entre o acelerômetro e o microcontrolador, ou seja, ligaram-se as saídas GND e 5V do acelerômetro aos pins GND e 5V do *mikro BUS 2* da placa *Curiosity HPC*, respetivamente. O *mikro BUS 2* é totalmente compatível com o *PIC18F47Q10* de 40 pins. As saídas do acelerômetro correspondentes aos eixos x, y e z, já filtradas, ligaram-se aos pins RA1, RA2 e RA3 do microcontrolador, respetivamente.

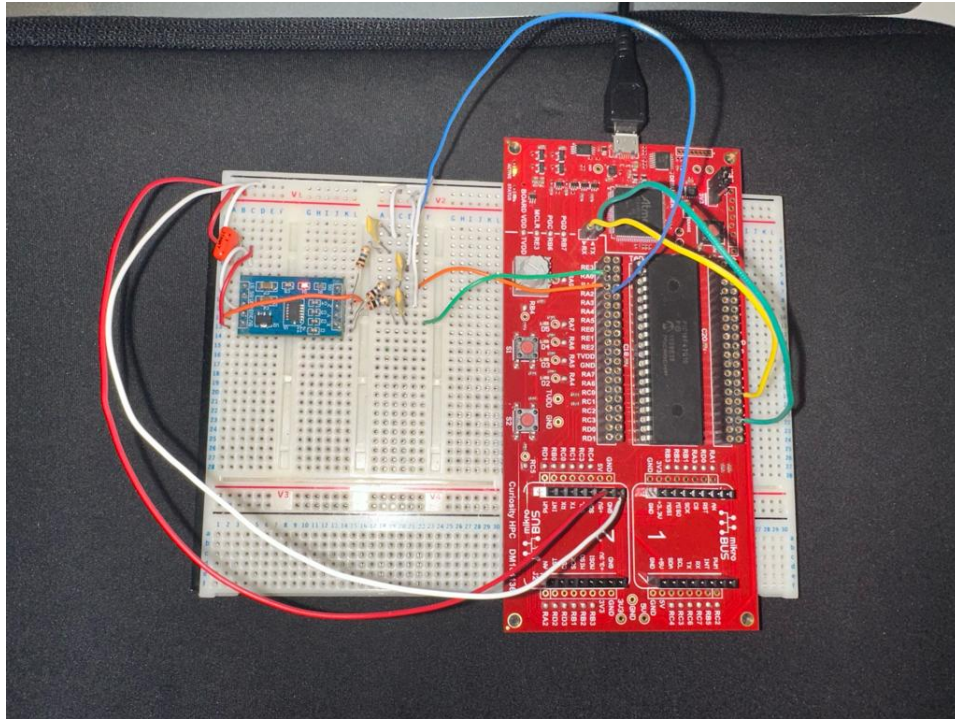


Figura 1: *Hardware* e ligações do acelerómetro ao microcontrolador.

A NCAP (*Network Capable Application Processor*) é composta pela aplicação, concebida utilizando a ferramenta *App Designer* do *software MATLAB* para apresentar os resultados obtidos e promover a interação entre o utilizador e o sistema.

A comunicação entre a TIM e o NCAP foi efetuada utilizando um cabo USB, tendo sido utilizada a USART *Universal Synchronous Asynchronous Receiver Transmitter*, que é uma interface de comunicação série, e que foi programada através do protocolo RS-232. Foi implementada uma comunicação bidirecional *full-duplex*, pelo que se tornou necessário fazer uso do TX (*transmitter*), que permite o envio de dados do PIC; do RX (*receiver*), que permite a receção de dados para o PIC; e do *ground*. Na Figura 1 pode também verificar-se, no próprio PIC, a ligação do pin TX ao pin RA4 e do pin RX ao pin RC7.

A norma IEEE Std. 1451 também foi incorporada e pretende facilitar a configuração e o controlo da TIM e, ainda, facilitar a leitura e escrita dos dados.

2.0.1 Diagrama de Blocos

Na Figura 2 é exibido o diagrama de blocos referente ao sistema desenvolvido com o propósito de destacar cada um dos componentes necessários, como também a inter-

relação entre os mesmos.

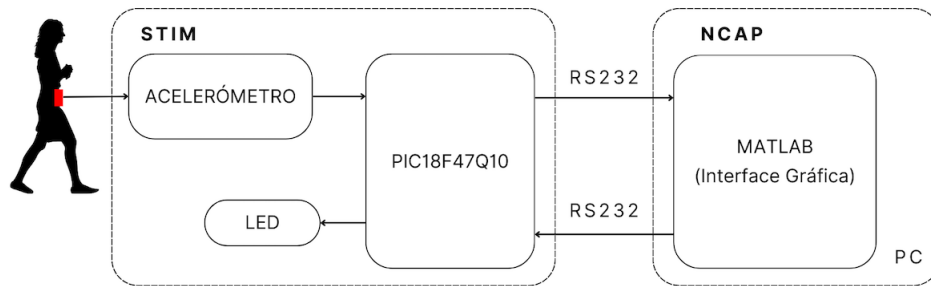


Figura 2: Diagrama de Blocos referente ao Projeto Desenvolvido.

Pela análise do diagrama, pode constatar-se que a STIM é composta pelo acelerômetro, pelo PIC18F47Q10 e pelo LED que corresponde ao atuador. Já a NCAP é constituída pela interface gráfica que foi elaborada na ferramenta *App Designer* do *MATLAB*. Como referido anteriormente, a ligação entre a STIM e a NCAP é feita através de um cabo USB que segue o protocolo RS232 e a comunicação entre ambos segue a norma IEEE Std. 1451.

2.0.2 Fluxograma

Na Figura 3 é exibido o fluxograma referente ao sistema desenvolvido com o propósito de fornecer uma representação mais detalhada do sistema, evidenciando as etapas sequenciais, decisões, ramificações e a lógica por trás das operações.

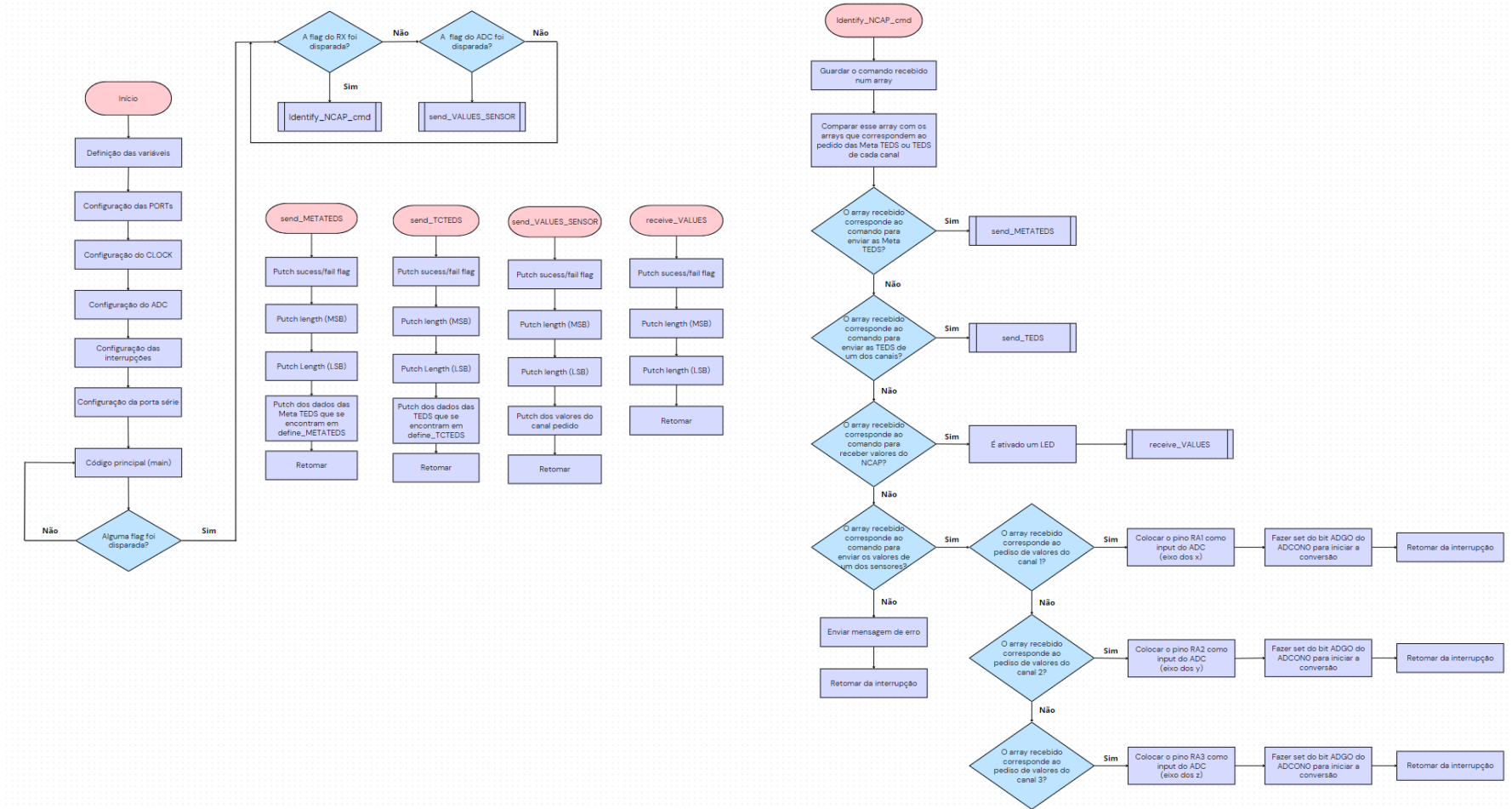


Figura 3: Fluxograma referente ao projeto desenvolvido.

2.0.3 Fluxograma da Interface da Gráfica

Na Figura 4 é exibido o fluxograma referente à interface desenvolvida com o propósito de fornecer uma representação mais detalhada da aplicação, evidenciando as etapas sequenciais, decisões, ramificações e a lógica por trás das operações.

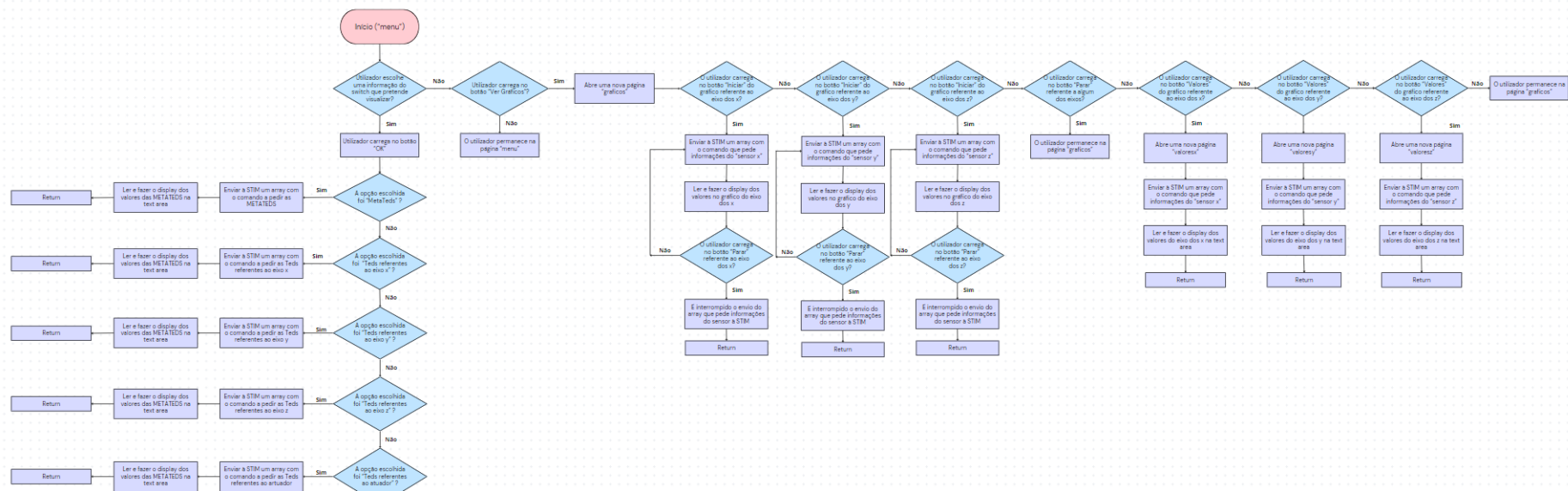


Figura 4: Fluxograma referente à Interface Gráfica desenvolvida.

3 Programação do Microcontrolador

3.1 *config*

No ficheiro *config.asm* é onde se realizou as configurações dos PORTs, do *clock*, do USART, do ADC e das interrupções. Nesse ficheiro, começou-se por estabelecer qual o microcontrolador que seria utilizado através do comando *#include*. Desta forma, incluiu-se o ficheiro que permite ao compilador reconhecer as instruções utilizadas para configurar o microcontrolador. De seguida, definiu-se a função global *_config*, que contém as configurações estabelecidas para o funcionamento do microcontrolador.

Foram também configurados alguns parâmetros com o comando *CONFIG* e com base no *datasheet* do próprio *PIC18F47Q10* [2]. O *external oscillator* (FEXTOSC) foi configurado para desativar a fonte de *clock* externa, permitindo que se possa escrever no pin RA7. O *clock switch enable* (CSWEN) foi configurado de forma a permitir que os valores de *New Oscillator Source* (NOSC) e de *New Divider Selection Request* (NDIV) possam ser alterados. Estes dois selecionam, respetivamente, a fonte e a frequência do *clock* que são usadas para a CPU e para os periféricos. Por fim, o *timer* do *watchdog* (WDT) foi configurado para que esteja desativado (OFF) impedindo que o microcontrolador estivesse constantemente a reiniciar. De seguida, apresentam-se as configurações dos PORTs. Para o presente caso apenas foram utilizados os pins referentes aos PORTs A e C, pelo que apenas estes PORTs foram configurados, tendo sido cada um deles configurado para três dos oito registos que possui: ANSELx, TRISx e LATx. O registo ANSELx permite configurar cada pin como digital ("0") ou analógico ("1"). O registo TRISx permite configurar cada pin como pin de *input* ("1") ou pin de *output* ("0"). O registo LATx permite alterar os valores do PORT, podendo afirmar-se que escrever no registo LATx tem o mesmo efeito que escrever no registo PORTx, contudo, enquanto que uma leitura do registo PORT corresponde ao valor I/O real, uma leitura do registo LATx corresponde ao valor do LATx e não ao valor presente nos I/O ports.

O registo BANKSEL é usado no decorrer do código para selecionar o banco de dados onde se encontra o registo que pretendemos manipular.

3.1.1 Configuração PORTA

Na configuração do PORTA, começou-se por colocar o valor de todos os seus pins a zero através do registo LATA, utilizando a instrução CLRF. De seguida, configurou-se os pins RA1, RA2 e RA3 como sendo *input*, através do registo TRISA ('0b00001110'), e analógico, através do registo ANSELA ('0b00001110'), já que estes foram os pins utilizados para conectar o microcontrolador aos três canais do acelerómetro. Os restantes pins do PORTA ficaram configurados como sendo *output* e digital.

3.1.2 Configuração PORTC

Na configuração do PORTC, começou-se igualmente por colocar o valor de todos os pins a zero através do registo LATC, utilizando a instrução CLRF. De seguida, configurou-se todos os pins como sendo *output*, através do registo TRISC, e digital, através do registo ANSELC, à exceção do pin RC7 que foi configurado como *input*.

Posteriormente, o comando RC4PSS foi colocado a '0x09', permitindo que seja estabelecida a EUSART1 (TX/CK) no pin RC4 (estabelecer RC4 como *transmitter pin* (TX)), enquanto que o comando RX1PPS foi colocado a 0b00010111 para estabelecer o pin RC7 como *receiver Pin* (RX). Estas configurações permitem a transmissão e receção de dados na comunicação série entre o microcontrolador e o computador.

3.1.3 Configuração CLOCK

Na configuração do CLOCK, começou-se por definir a frequência do *clock* a 32MHz, recorrendo-se para tal ao registo OSCFREQ ('0b00001110'). Esta frequência de 32MHz corresponde à *High-Frequency Internal Oscillator* (HFINTOSC). De seguida, passou-se por definir os valores do NOSC e do NDIV através do registo OSCCON1. Este registo define a fonte de oscilação interna e o divisor para o *clock* que são usados para a CPU e para os periféricos. O valor do NOSC foi definido consoante a frequência da fonte do relógio (HFINTOSC) desejada, ficando, por isso, NOSC = 0110. A fim de que a frequência do *clock* fosse equivalente ao valor de HFINTOSC, colocou-se NDIV = 0000, o que significa que o *clock* é dividido por 1. Assim, a frequência do *clock* é de 32MHz. Os osciladores devem ser ativados pelo registo *Oscillator Manual Enable Register* (OSCEN) e, deste modo, ativou-se o HFINTOSC ('0b01000000').

3.1.4 Configuração USART

Na comunicação série estabelecida entre o microcontrolador e o computador é usada a EUSART (*Enhanced Universal Synchronous Asynchronous Receiver Transmitter*) e, por isso, para a configuração da porta série foram utilizados os registos da EUSART1.

Começou-se primeiramente por definir o *baud rate*, correspondendo este à taxa com que a porta série envia informação. Neste caso, o *baud rate* é equivalente ao débito binário que, por sua vez, é definido pelo produto *frequência de amostragem* $\times n^o$ de bits. Tendo-se uma frequência de amostragem de 160Hz por canal, tem-se uma frequência de amostragem total de 480Hz.

Sendo a mensagem enviada pela comunicação série uma mensagem de 8 bits, contabilizando ainda o *start bit* e o *stop bit*, no total tem-se 10 bits, de cada vez, a ser enviado pela comunicação série. Além disso, cada *array* correspondente a *output* possui no máximo 7 *bytes*. Correspondendo a estrutura da resposta da STIM ao pedido da NCAP a 1 *byte* para o *Success/Fail Flag*, 2 *bytes* para o *Length LSB* e *Length MSB* e 1 *byte* para o valor lido, tem-se 4 *bytes* no total. Assim, resulta em 110 *bits* por amostra (7 *bytes* $\times$ 10 *bits* + 4 *bytes* $\times$ 10 *bits*).

Posto isto, o débito binário corresponde a $480 \times 110 = 52800$ bits/segundo, pelo que o *desired baud rate* seria de 52800 para a transmissão de dados. Um valor superior também seria suficiente.

$$DesiredBaudrate = \frac{F_{osc}}{(64 * SPxBRG) + 1}$$

$$SPxBRG = \left(\frac{F_{osc}}{(64 * DesiredBaudrate)} \right) - 1$$

Substituindo F_{osc} por 32MHz e o *desired baud rate* por 52800, obteve-se um valor, em decimal, de SPxBRG igual a 8.470, ou seja, aproximadamente a 8.

Assim, como SP1BRG é de 8, colocou-se este valor no SP1BRGL (*byte* menos significativo), enquanto que no SP1BRGH (*byte* mais significativo) colocou-se todos os bits a zero, já que só vai ser usado apenas um dos EUSART disponíveis.

De seguida, no registo TX1STA (registo que permite transmitir dados), ativou-se a transmissão, definiu-se o número de bits a enviar (8 bits) e, sendo a comunicação

em série RS232 assíncrona, configurou-se a transmissão EUSART para o modo assíncrono. Assim, o registo TX1STA ficou definido como '0b10100000'. Por fim, no registo RC1STA (registo que permite receber dados), ativou-se a receção contínua, definiu-se o número de bits a receber (8 bits) e ativou-se, ainda, a porta série. O registo RC1STA ficou definido como '0b10010000'.

3.1.5 Configuração ADC

Na configuração do ADC, começou-se por definir qual o pin que se pretendia que estivesse associado ao canal de entrada positivo do ADC. Como tal, definiu-se o registo ADPCH (*ADC Positive Input Channel Selection*) como 000001, ou seja, o ADC está ligado ao pin RA1, correspondendo este pin ao eixo dos x do acelerómetro. Este registo terá de ser alterado consoante o eixo do qual se pretende ler o sinal.

De seguida, no registo ADREF (*Reference Selection Register*) definiu-se a tensão de referência positiva do ADC conectada ao V_{DD} e a tensão de referência negativa do ADC conectada ao V_{SS} . Ficando este registo definido como 00000000.

Tendo em vista uma frequência de conversão do ADC de 1MHz de modo a que o tempo de conversão não influencie o tempo de amostragem, e sabendo que $F_{osc} = 32\text{MHz}$, foi possível configurar o registo ADCLK (*ADC Clock Selection Register*) através do valor de n , que foi calculado através da seguinte expressão.

$$ADCfreq = \frac{F_{osc}}{2(n+1)}$$

$$n = 15$$

Assim, no registo ADCLK foi colocado o valor '0b00001111'.

Por fim, configurou-se o registo ADCON0 (*ADC Control Register 0*). Começou-se por fazer *enable* ao ADC e por definir que por cada *trigger* de conversão completo o ADC é limpo. Além disso, definiu-se que o *clock* é alimentado pelo F_{osc} e que os resultados do ADC estarão justificados à esquerda, estabelecendo que serão guardados os 8 bits mais significativos separados dos dois bits menos significativos. Assim, no registo ADCON0 foi colocado o valor '0b1000000'.

3.1.6 Configuração das Interrupções

As interrupções são eventos que interrompem momentaneamente a normal execução do sistema de forma a que este lide com esses eventos externos, não comprometendo o programa na sua totalidades. No presente trabalho, fez-se uso da interrupção do *Timer0*. Fez-se uso também da interrupção do ADC que será ativada quando a conversão de um dos canais tiver terminado. E, ainda, da interrupção do RX é ativada quando o microcontrolador tiver recebido um *byte* pela porta-série.

Começou-se, assim, por configurar os registos PIRx (*Peripheral Interrupt Request (Flag)*) com o objetivo de colocar as *flags* das interrupções a '0'. Assim, os *Interrupt Flag bit*, isto é, o bit 5 do registo PIR0, o bit 0 do registo PIR1 e o bit 5 do registo PIR3 foram colocados a '0', que correspondem, respetivamente, às *flags* do *Timer0*, do ADC e do RX1. De seguida, fez-se a ativação (*enable*) das interrupções, configurando-se para isso os registos PIRx (*Peripheral Interrupt Enable*). De forma análoga, os *Interrupt Enable bit*, isto é, o bit 5 do registo PIE0, o bit 0 do registo PIE1 e o bit 5 do registo PIE3 foram colocados a '1', que correspondem, respetivamente, às *flags* do *Timer0*, do ADC e do RX1. Posteriormente, foi realizada a ativação tanto do *Timer0*, quanto do ADC. Assim, colocou-se o bit 7 do registo T0CON0 e o bit 7 do registo ADCON0 ambos a '1'. Por fim, foi ainda necessário fazer o *enable* das interrupções periféricas e globais, colocando a 1 os bits 6 e 7 do registo INTCON, respetivamente.

3.2 *main*

O ficheiro *main.c* constitui o ficheiro que contém o código principal que é executado quando se executa o programa. Por conseguinte, neste ficheiro são declaradas todas as variáveis e funções globais elaboradas e, ainda, são efetuados alguns *includes*, a fim de uma correta programação do microcontrolador e, consequentemente, uma correta execução do programa.

De seguida, apresentam-se as funções desenvolvidas neste ficheiro.

3.2.1 *main*

A função *main* é o ponto de partida para a execução do programa, começando por realizar as configurações definidas no ficheiro *config*. As funções *define_METATEDS* e *define_TCTEDS* são também evocadas, tendo estas a função de definir e preencher as estruturas de dados acerca das TEDS (*Transducer Electronic Data Sheets*) e das METATEDS. É, ainda, criado um *loop while infinito*, fazendo com que o pro continuamente.

Esta é a função principal de todo o código, sendo efetuada quando o programa se encontra no seu fluxo normal, sem que ocorra nenhuma interrupção. Quando ocorre uma interrupção, o programa é interrompido, sendo corrida a função *interrupt_handler*, que verifica qual a interrupção que ocorreu.

De notar que é uma função que não recebe nenhum argumento, nem retorna nenhum valor, assim como as funções *define_METATEDS* e *define_TCTEDS*, que são explicadas de seguida.

3.2.2 *define_METATEDS*

A função *define_METATEDS* é a função que define as informações sobre as METATEDS. As METATEDS são o *datasheet* de todo o sistema e, dessa forma, abrangem informações importantes sobre o sistemas e os transdutores que o constituem. Estas são requeridas por qualquer STIM e são importantes para permitir uma comunicação mais eficiente acerca de informações sobre os transdutores entre diferentes equipamentos eletrónicos.

A função *define METATEDS* é constituída por três blocos de dados, TEDS ID,

UUID e Max Channels. O TEDS ID inclui informações gerais sobre o sistema, o UUID é um identificador único do dispositivo em questão e o Max Channels dá o número de canais do sistema.

Cada um destes blocos corresponde a um *array* onde o primeiro valor corresponde ao *field type* que é o identificador desse bloco, o segundo ao tamanho do *array* e os valores seguintes correspondem a informação. Cada um desses valores foi definido tendo em conta a norma IEEE Std.1451, [3]. É importante referir que o último valor do *array* correspondente ao *Max Channels* é '4', uma vez que se considerou que este sistema tem quatro transdutores (três correspondem aos eixos x, y e z do acelerómetro, embora se tenha somente um sensor, e um corresponde ao atuador).

3.2.3 *define_TCTEDS*

De forma semelhante ao que foi feito na função *define_METATEDS*, criou-se uma função, denominada (*define_TCTEDS*), que contém informação sobre cada um dos transdutores do sistema (TEDS). Desta forma, para cada canal existem oito blocos: TEDSID, CHANNEL TYPE, UNITS, LOW RANGE LIMIT, HIGH RANGE LIMIT, DATA MODEL, DATA LENGTH e MODEL SIG BITS.

Tal como nas METATEDS, cada um destes blocos corresponde a um *array* onde o primeiro valor corresponde ao *field type* que é o identificador desse bloco, o segundo ao tamanho do *array* e os valores seguintes correspondem a informação.

Este sistema apresenta três "sensores", um para cada eixo do acelerómetro, sendo que o TCTED1 corresponde ao eixo x, o TCTED2 ao y e o TCTED3 ao z. No entanto, sendo estes canais do mesmo sensor, as informações das TEDS serão as mesmas. No entanto, é importante referir alguns dos valores. Nos blocos *TEDS Low/High range limits* foi definido '0' como o valor mínimo e '255' para o valor máximo que os "sensores" podem ler. Para o atuador foram definidos os mesmos valores. Para os TCTEDS referentes aos "sensores", o bloco *UNITS* foi definido para m/s^2 , pois os valores correspondem a valores de aceleração. Relativamente ao atuador, o bloco *UNITS* foi definido para *volts*, uma vez que se trata de um LED. Existe uma forma específica para definir as unidades SI, servindo de exemplo o caso m/s^2 . Todas as unidades que não fossem *metro* e *segundo* foram definidas como '128'. Já a unidade *metro*, cujo expoente é '1', tomou o valor de $(1*2)+128=130$. A unidade *segundo*, cujo expoente é '-2', tomou o

valor de $(-2*2)+128=124$.

3.2.4 *identify_NCAP_cmd*

Esta função tem como objetivo receber o comando vindo da NCAP, decodificá-lo e decidir o que fazer tendo em conta o comando recebido. A NCAP pode pedir informação acerca das METATEDS, das TEDS dos três "sensores" e do atuador ou, ainda, informações sobre os valores adquiridos por parte de cada um dos três "sensores". Desta forma, dependendo do pedido que é feito, uma mensagem específica é enviada para a STIM, de forma a que se consiga entender qual é a informação que a NCAP pretende obter.

De forma a que o microcontrolador perceba qual a mensagem requisitada, foram definidos inicialmente oito *arrays* que correspondem às mensagens enviadas pela NCAP quando faz um pedido. Foi também inicializado o *array cmd1* onde será guardado o comando que vem do NCAP.

De seguida criou-se um ciclo *while*, que corre até a mensagem ser totalmente recebida, que verifica se foi recebido algum carater através da ativação da *flag* RC1IF. Se esta situação se verificar, o carater é armazenado no *array cmd1*, incrementando o índice a cada iteração.

Foi também criado um *array* chamado *dados* que armazena os dados recebidos do NCAP. É também definido com todos os valores como 0 e possui um tamanho de 256.

Os blocos *if* criados comparam os *arrays cmd1* e *dados* com os *arrays* já definidos (como METATEDS, TCTEDS1, etc.). Quando encontra uma correspondência, chama funções específicas (*send METATEDS()*, *send TCTEDS()*, *send VALUES SENSOR()* ou *receive VALUES()*) tendo em conta a correspondência encontrada.

É importante referir que no caso do *array* recebido corresponder ao *array atuador* para além de ser chamada a função *receive VALUES()*, o LED 6 acende. Este *array* em específico é enviado pela NCAP quando o valor da aceleração do utilizador é superior a 23.

3.2.5 *send METATEDS*

A função *send_METATEDS* começa por enviar uma mensagem como resposta da STIM ao comando enviado pela NCAP. Através da função *putch*, são enviados os valores '1', '0' e '22' que, segundo a norma, dizem respeito respetivamente à *flag* de sucesso, ao tamanho dos MSB e ao tamanho dos LSB (número de *bytes* que serão enviados após a terceira evocação ao *putch*). Esses *bytes* correspondem à informação especificada na função *define_METATEDS*.

Esta função não recebe nenhum argumento, nem retorna nenhum valor.

3.2.6 *send TCTEDS*

De forma análoga, a função *send_TCTEDS* evoca três vezes a função *putch*, sendo enviados os valores '1', '0' e '36', que correspondem também à *flag* de sucesso, ao tamanho dos MSB e ao tamanho dos LSB, respetivamente. E, de seguida, são enviados os respetivos *bytes* que correspondem à informação especificada na função *define_TEDS*, consoante a TED pedida.

Contudo, como existe mais do que uma TED, tornou-se essencial que a função *send_TCTEDS* recebesse argumento e, como tal, criaram-se variáveis no formato *uint8_t* para cada uma das TEDS. Estas definições são realizadas no início do ficheiro *main.c*. Quando a NCAP pede à STIM para ler uma TED, entra como argumento na função *send_TCTEDS* a respetiva variável em formato *uint8_t*.

3.2.7 *send VALUES_SENSOR*

A função *send_VALUES_SENSOR* é evocada sempre que é enviado um comando pelo NCAP a requisitar os valores dos sensores.

Como o sistema tem 3 sensores dos quais podem ser pedidos valores, dentro desta função foram criados 3 blocos, sendo que cada um deles corresponde ao que é executado para cada canal pedido pelo NCAP. Assim, para cada canal o ADPCH é atualizado para que o input do ADC corresponda ao canal de onde são pretendidos os valores. É importante referir que foram definidos 3 canais associados aos pinos RA1, RA2 e RA3 que correspondem aos inputs de cada um dos sensores.

Posteriormente é iniciada a conversão do ADC e aguarda-se até que esta esteja

concluída. Quando essa condição se verifica a *flag* do ADC é desativada é feito o *putch* do valor que se encontra no banco de dados ADRESH. No final, os valores desse registo são apagados de forma a que seja possível guardar novas conversões do ADC.

3.2.8 *receive_VALUES*

A função *receive_VALUES* é chamada quando é recebido um comando vindo da NCAP que tem como objetivo enviar informações que permitam saber se o LED foi ligado com sucesso. Assim, é feito o *putch* dos valores '1', '0' e '0', que correspondem à *flag* de sucesso, ao tamanho dos MSB e ao tamanho dos LSB, respetivamente.

3.2.9 *putch*

A função *putch* tem como objetivo fazer o envio de um *byte* pela porta série.

Primeiramente, a função aguarda até que a *flag* de interrupção *TX1IF* seja definida como '1'. Essa *flag* indica se é possível enviar dados pela porta série. Quando é '0', significa que o *buffer* de transmissão está cheio e que, por isso, ainda não é possível enviar novos dados. Quando a *flag* de interrupção *TX1IF* está ativa, o *byte* que se pretende enviar é transmitido para *TX1REG* (porta série). Finalmente, espera-se pelo fim do envio desse *byte* que acontece quando a *flag* *TX1IF* está ativa (definida como '1'). Neste momento, é novamente possível enviar mais *bytes* pela porta série.

3.2.10 *getch*

A função *getch* tem como objetivo ler um carater recebido pela porta série.

Quando a porta série recebe um carater pela UART, este é armazenado no registo RC1REG. E a função *getch* atribui o valor que se encontra em RC1REG à variável *get char*, que é uma variável global onde se armazena o carater recebido.

3.3 *interrupt_handler*

No ficheiro *interrupt_handler.asm* ocorre o desencadeamento de quando ocorre uma interrupção. Dessa forma, a principal finalidade do código desenvolvido é a de, através da verificação da *flag* que foi ativada, decodificar qual a interrupção que foi acionada.

Através da instrução BTFSC (*Bit Test File, Skip if Clear*) verifica-se o estado do bit de um registo, podendo este corresponder à *flag* do ADC ou do RX. A instrução BTFSC permite fazer *skip* para outra instrução quando o estado do bit é '0'.

Antes dessa verificação, são declaradas as seguintes funções globais: *int_handler*, *getch*, *identify_NCAP_cmd*, *adc_int_handler*.

3.3.1 *Interrupção do ADC*

Assim, no caso da interrupção do ADC, verifica-se o estado do bit 0 do registo PIR1. Caso esse bit esteja a '1', ocorreu uma interrupção do ADC e o fluxo de execução do programa é direccionado para o *adc_int_handler*. Caso contrário, faz (*skip*). No *adc_int_handler* começa-se por seleccionar o registo ADRESH (*ADC Result Register bits. High bits*) que guarda os oito bits mais significativos do resultado da conversão do ADC. De seguida, faz-se *clear* à *flag* da interrupção do ADC colocando o bit 0 do registo PIR1 a '0'. Por fim, usa-se o comando RETFIE para retornar ao código principal.

3.3.2 *Interrupção do RX*

Igualmente para a interrupção do RX, verifica-se o estado do bit 5 do registo PIR3. Caso esteja a '1', ocorreu uma interrupção do ADC e o programa chama a função *identify_NCAP_cmd*, que verifica o comando enviado pela NCAP. Caso contrário, faz (*skip*). Trata-se de uma interrupção cuja *flag* não pode ser limpa pelo *software*, pois o respetivo bit é somente de leitura. Por fim, utiliza-se o comando RETFIE que é usado para fazer o retorno ao código principal após o tratamento da interrupção.

3.4 *TEDS definition*

O ficheiro *Teds_definition* é um ficheiro *.h* onde estão definidas as estruturas das METATEDS e das TEDS.

No caso das METATEDS definiu-se a sua estrutura como sendo composta por três *arrays*: TEDSID, UUID, MAXCHAN. O valor entre parêntesis retos à frente de cada um deles representa o tamanho do *array*.

No caso das TEDS definiu-se a sua estrutura com sendo composta por oito *arrays*: TEDSID, CHANNEL TYPE, UNITS, LOW RANGE LIMIT, HIGH RANGE LIMIT, DATA MODEL, DATA MODEL LENGTH e MODEL SIG BITS. Mais uma vez, o valor entre parêntesis retos à frente de cada um deles representa o seu tamanho.

4 Interface

Por fim, foi elaborada uma interface, com recurso ao *software Matlab* utilizando a ferramenta *App Designer*, a fim de apresentar os resultados obtidos e de promover a interação entre o utilizador e o sistema.

Para que a correta execução da interface foi elaborado o ficheiro *Comandos_STIM* no *software MATLAB*. Assim, nesta interface é possível ter acesso às METATEDS e TEDS, assim como visualizar e verificar os valores de aceleração de cada um dos eixos individualmente em tempo real.

Nesse sentido, a página principal da interface começa por proporcionar ao utilizador, através de um componente *Drop Down*, a escolha de umas das seguintes opções: "*MetaTeds*", "*Teds referentes ao eixo do x*", "*Teds referentes ao eixo do y*", "*Teds referentes ao eixo do z*" e "*Teds referentes ao atuador*". Depois de escolhida uma destas opções e depois de carregar no botão "*OK*", aparece numa *Text Area* a informação correspondente à opção selecionada. Por fim, esta página apresenta um botão "*Gráficos*", que permite acesso a uma segunda página.

Na Figura 5 está apresentada a página principal, tendo sido, a título de exemplo, selecionada a opção "*Teds referentes ao eixo do y*", podendo verificar-se a respetiva informação na *Text Area*.



Figura 5: Página "menu".

Quando é selecionada uma opção no componente *Drop Down* e, de seguida, pressionado o botão "*OK*", a função *Comandos_STIM* é evocada, tendo como argumento uma das opções possíveis ('1', '2', '3', '4' ou '5').

A página que surge após o botão "*Gráficos*" ser pressionado, apresenta um conjunto de três gráficos, estando cada um deles relacionado a um dos eixos do acelerómetro. Tal pode verificar-se na Figura 6.

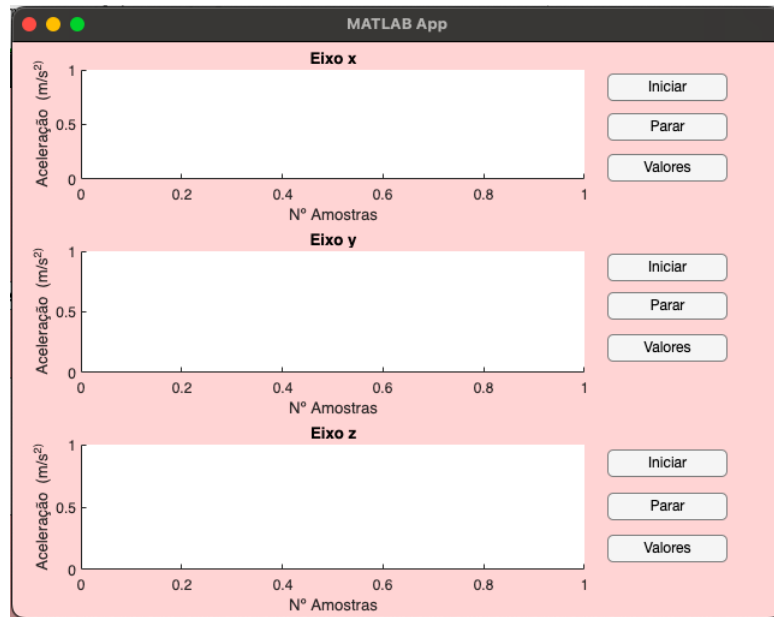


Figura 6: Página "graficos".

Estes gráficos têm o intuito de apresentar a variação dos valores de aceleração por número de amostras. Para isso, a cada gráfico estão associados os seguintes botões: "*Iniciar*", "*Parar*" e "*Valores*".

Quando o primeiro é pressionado, a função *Comandos_STIM* é evocada, tendo como argumento a opção desejada. Por exemplo, se se pressionar o botão "*Iniciar*" associado ao gráfico referente ao eixo do x, a função *Comandos_STIM* é evocada, tendo como argumento a opção 6. Por conseguinte, com os valores que advêm dessa função, é feito o plot gráfico, podendo visualizar-se a variação dos valores de aceleração para o eixo escolhido. Por outro lado, o botão "*Parar*" faz com que o botão "*Iniciar*" fique desativado e, por isso, deixa de se poder visualizar a variação dos valores de aceleração. Já o botão "*Valores*" abre uma nova página. Como exemplo, na Figura 7 é visível a página "valoresx", que permite ao utilizador conferir os valores de aceleração para o eixo em questão.



Figura 7: Página "valoresx".

5 Resultados e Discussão

O teste do funcionamento do sistema desenvolvido começou por se pedir à NCAP para visualizar as informações referentes ao canal das METATEDS. Na Figura 8 pode verificar-se o resultado obtido.

```
Qual a opção: 1
Os dados relativos às Meta TEDS são os seguintes:
Meta TED ID:      3      4      0      1      0      1

Meta TED UUID:     4     10      0      0      0      0      0      0      0      0      0      1

Meta TED Max Channels:  13      2      0      4
```

Figura 8: Resultado da resposta da STIM ao pedido da NCAP para ler as METATEDS.

De igual forma, pediu-se à NCAP para visualizar as informações referentes aos canais das TEDS dos três "sensores". Tal é visível nas Figuras 9, 10 e 11.

```

Qual a opção: 2
Os dados relativos às TEDS do canal 1 são os seguintes:
TransducerChannel 1 TEDS ID:      3      4      0      3      0      1

TransducerChannel 1 TEDS Channel Type:    11      1      0

TransducerChannel 1 Units:      12      10      0      0      0      130      0      132      0      0      0      0

TransducerChannel 1 TEDS Low range limit:    13      1      0

TransducerChannel 1 TEDS High range limit:    14      1      255

TransducerChannel 1 TEDS Data model:      40      1      0

TransducerChannel 1 TEDS Data model length:    41      1      1

TransducerChannel 1 TEDS Data model significant bits:    42      1      8

```

Figura 9: Resultado da resposta da STIM ao pedido da NCAP para ler as TCTEDS referentes ao eixo dos x.

```

Qual a opção: 3
1      0      36

Os dados relativos à Ted do canal 2 são os seguintes:
TransducerChannel 2 TEDS ID:      3      4      0      3      0      1

TransducerChannel 2 TEDS Channel Type:    11      1      0

TransducerChannel 2 Units:      12      10      0      0      0      130      0      132      0      0      0      0

TransducerChannel 2 TEDS Low range limit:    13      1      0

TransducerChannel 2 TEDS High range limit:    14      1      255

TransducerChannel 2 TEDS Data model:      40      1      0

TransducerChannel 2 TEDS Data model length:    41      1      1

TransducerChannel 2 TEDS Data model significant bits:    42      1      8

```

Figura 10: Resultado da resposta da STIM ao pedido da NCAP para ler as TCTEDS referentes ao eixo dos y.

```

Qual a opção: 4
1      0      36

TransducerChannel 3 TEDS ID:      3      4      0      3      0      1

TransducerChannel 3 TEDS Channel Type:    11      1      0

TransducerChannel 3 Units:      12      10      0      0      0      130      0      132      0      0      0      0

TransducerChannel 3 TEDS Low range limit:    13      1      0

TransducerChannel 3 TEDS High range limit:    14      1      255

TransducerChannel 3 TEDS Data model:      40      1      0

TransducerChannel 3 TEDS Data model length:    41      1      1

TransducerChannel 3 TEDS Data model significant bits:    42      1      8

```

Figura 11: Resultado da resposta da STIM ao pedido da NCAP para ler as TCTEDS referentes ao eixo dos z.

Posteriormente, solicitou-se à NCAP a visualização do gráficos referentes às três componentes da aceleração, tal resultado pode verificar-se nas Figuras 12, 13 e 14.

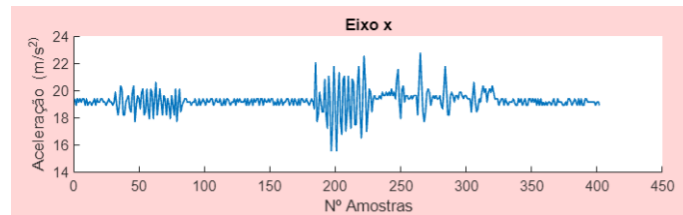


Figura 12: Gráfico referente à componentes x da aceleração, como resposta da STIM ao pedido a NCAP.



Figura 13: Gráfico referente à componente y da aceleração, como resposta da STIM ao pedido a NCAP.

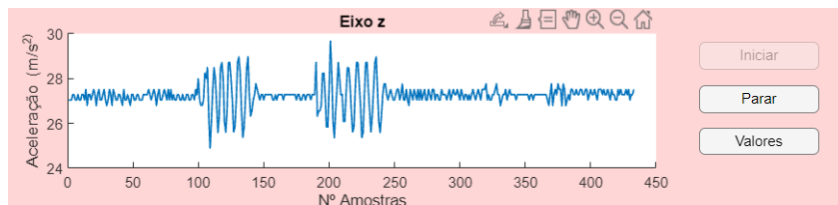


Figura 14: Gráfico referente à componente z da aceleração, como resposta da STIM ao pedido a NCAP.

A comparação direta com os sinais obtidos no osciloscópio não foi possível de realizar, pelo que não foi possível verificar se a amostragem se sucedeu corretamente.

Por fim, solicitou-se à NCAP para exibir os valores de cada uma das componentes da aceleração em tempo real. Na Figura 15 está exibido o respetivo resultado para a componente .

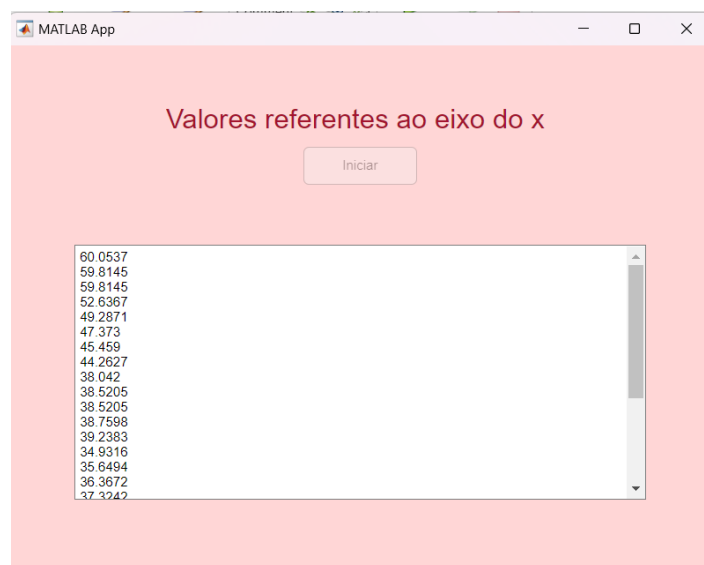


Figura 15: Valores referentes à componente x da aceleração como pedido da STIM à NCAP.

6 Conclusão

Este projeto teve-se como principal objetivo o desenvolvimento de um sistema de aquisição e processamento de sinais provenientes de um acelerómetro durante o movimento de marcha do utilizador.

Posto isto, incluiu-se um acelerómetro *MMA7361L* e uma placa *Curiosity HPC* onde está inserido um microcontrolador *PIC18F47Q10*. A comunicação entre o PIC e o computador foi efetuada utilizando a USART, interface de comunicação série, sendo estabelecida através do protocolo *RS232* e foi aplicada a norma IEEE Std. 1451.

A programação do microcontrolador foi conseguida com recurso ao *software MPLAB*, utilizando tanto linguagem *C* como linguagem *Assembly*. Em *MATLAB* foi desenvolvido o algoritmo capaz de receber e enviar dados ao microcontrolador. Foi, ainda, concebida uma aplicação utilizando a ferramenta *App Designer* do *software MATLAB* para apresentar os resultados obtidos e promover a interação entre o utilizador e o sistema.

Uma possível melhoria a implementar no projeto poderia passar pela implementação de filtros passa-baixo aos sinais adquiridos pelo acelerómetro, a fim de diminuir o ruído dos mesmos. Aditivamente, poderia proporcionar-se ao utilizador uma visualização gráfica dos sinais dos três eixos do acelerómetro em simultâneo. Além disso, outra melhoria poderia também passar por uma otimização da ativação do LED no sentido de este ser ativo apenas quando um dado valor de aceleração é ultrapassado e não ser ficar acionado definitivamente após a primeira ocorrência. Adicionalmente, poderia realizar-se, na interface, a decodificação dos valores utilizados para definir as METATEDS e TEDS, tornando essa informação mais clara para o utilizador.

Referências

- [1] MOTOROLA. JFET Input Operational Amplifiers TL081C,AC, TL082C,AC, TL084C,AC Datasheet.
- [2] Microchip Technology Inc. 28/40/44-pin, Low-Power, High-Performance Microcontrollers, PIC18F27/47Q10 Datasheet.
- [3] IEEE Standard for a Smart Transducer Interface for Sensors and Actuators - Common Functions, Communication Protocols, and Transducer Electronic Data Sheet (TEDS) Formats. *IEEE Std 1451.0-2007*, pages 1–335, 2007.